Name : AKASH SAMPATH

Topic : Virtualization and Containerization

Virtual Machine : The Infrastructure Layer :

- The infrastructure layer is the physical foundation that supports everything above it and it is the actual hardware with basic facilities that provide raw computing power , storage and network connectivity
- These are physical components where you can touch and interact and without this layer, the hypervisor , Guest OS and Apps wont exists and its like a software without a computer
- This layer is the actual physical machine and the foundation that provides everything the Virtual Machine need to exist

Inside the Infrastructure Layer :

- We have Physical Servers such as actual computers with CPU , RAM , Local disk and we have storage systems such as hard drive and networking systems such as switches , routers , cables and network interface which used for internet connectivity
- There are some main physical components in this layer such as the CPU where this does the processing and RAM is the short term memory where the computer keeps data it needs to access right now and the Storage is where al the data , OS files and apps are saved permanently and Networking components that allows the hardware to talk to the internet

Working of Infrastructure Layer :

- For Virtual Machine the hypervisor sits directly on the infrastructure layer and divides those physical resources into virtual ones allowing multiple guest OS to share the same hardware
- In Virtual Machine setup the hardware doesn't know its running multiple apps or operating systems
- And therefore the infrastructure is the bottom layer that delivers CPU , Memory and disk space and network connection to all the software layers above and with is layer the entire stack is built

Virtual Machine : Hypervisor Layer :

- The hypervisor is a software layer that sits between the physical hardware (infrastructure layer) and the virtual machines and its job is to filter (abstract) the hardware so that multiple operating systems can run on the same physical machine and each thinking it has its own dedicated hardware
- Also the hypervisor is known as a Virtual Machine Monitor which is the brain of the virtual machines (Virtualization) and it acts as a middle men between the physical hardware (infrastructure) and the multiple operating systems (Guest OS)
- This is also a software layer that sits directly on the physical hardware and tricks the operating system thinking they each have their own computer

Inside the Hypervisor Layer : Virtual Machine :

- Hypervisor manages the CPU and since multiple virtual machines want to use the processor at the same time , the hypervisor decides which VM gets to use the CPU for a few seconds and then swaps in the next one and ensures each Virtual Machines doesn't steal all the CPU from each other

- Hypervisor connects (maps) Virtual RAM to Physical RAM by ensuring that app1 cannot see or overwrite the memory being uses by app2

- The hypervisor mimics hardware components so that the guest OS can use them without knowing they are virtual and making each Virtual machine thinking it is real computer

Working of Hypervisor Layer : Virtual Machine :

- A Hypervisor sits between the physical hardware like computer's CPU , Memory and Storage and the virtual machines and shares the physical hardware resources among multiple Virtual Machines and making each VM think it has its own hardware
- Also it takes one physical machine (Infrastructure Layer) and makes it think it is independent computer
- It also sits between the physical hardware and the multiple virtual machines
- This also manages resources so multiple operating systems (Guest) can share one physical computer (Host)

So first : (Hardware Abstraction / Abstraction) :

- The hypervisor takes the real physical hardware like CPU crores , RAM , disk , and network components and creates a virtual versions of these components and also creating a fake virtual versions of these hardware parts for each VM and each VM thinks it has its own processor and memory and but it's just a small piece of real physical hardware
- Create 4 virtual CPUs from 2 physical CPUs and take 8GB of virtual RAM from 16GB physical RAM

Second : (Isolation) :

- Each VM runs its own operating system (Guest OS) and applications and the hypervisor ensures they don't interfere with each other and if one VM crashes the others keep running
- This is like a security wall and if app1 crashes or gets a virus , the hypervisor ensures it stays trapped inside its own box (app1) and it cannot reach over and affect app2 because the hypervisor keeps them completely separated
- As the hypervisor keeps each VM in its own space and if app1 or its guest OS crashes and the hypervisor ensures it doesn't affect app2 or the rest of the system

Third : (Resource Scheduling / Partitioning) :

- The hypervisor acts like a schedular when a VM needs to do some work like run a program and the hypervisor decides when and how much of the Physical CPU and memory to give
- It might give some CPU time to first VM and then switch to second VM and so on and it feels like they are all running at once
- Its uses the physical hardware and if your laptop has 16GB or RAM , the hypervisor might give 4GB to first VM (Guest OS 1) and 8GB to second VM (Guest OS 2) and ensures they don't fight over the same resources

Fourth : (Emulation) :

- The hypervisor creates a virtual hardware and the guest OS thinks is seeing a real hard drive and a real network component and actually they are just software files managed by the hypervisor so that the Guest OS doesn't need special drivers of software

In Virtual Machine : App1 Layer :

- In virtual machine the app1 layer represents a single application like a web server , database , or a custom program that is running inside that virtual machine and it's the actual software you want to use
- App1 layer is the top most level where your actual software lives and sitting directly on top of the libraries and the operating system used to that specific VM

Inside the App1 Layer : Based on actual software you want to use :

- Application code is the actual program files like python scripts and .exe files , java files
- Configuration files are the settings that tell the app how to behave
- Runtime data is temporary files or logs the app creates while running
- Dependencies are where the app needs specific libraries or framework but those are usually stored in the Bins/Lib layer

Based on building a web service :

- Source code is the complied code that runs your logic like .jar file for java and .py file for python or .exe for Windows
- Applications Configs are files like application.properties , .env file or XML configurations that tell the app which database to connect to and which port to use
- Static Assets are in the web app contains HTML , CSS , images and JavaScript files
- Local Data is where any temporary files or logs the application generates while its running

Based on Software Functionality :

- Source code is the actual logic and instruction of the program
- Application Configuration is the settings , environment variables and parameters that tell the app how to behave
- User Interface Components is when if it's a user facing app , the code that generates what you see on the screen

Working of App1 : Virtual Machine :

App1 runs on top of a Guest OS :

- Inside the VM , there is a full operating system like Windows or Linux called the Guest OS .
- The app is installed and runs just like it would on a physical computer it relies (depends) on the Guest OS to provide services like accessing files or using memory and communication through network

The Guest OS thinks its on real hardware :

- The Guest OS doesn't know its virtualized and it communicates with what it believes is the physical hardware like CPU , RAM and disk but actually those are virtual devices provided by the hypervisor

Isolation :

- App1 is completely isolated from other apps like app2 and app3 that might be running in separate VMs on the same physical machine
- Each VM has its own Guest OS and its own copy of libraries and binaries so app1 doesn't interfere with others

Self-Contained :

- When app1 runs it believes it has the entire computer itself and doesn't know that app2 or app3 exist
- This is because the Guest OS (the operating system inside the specific VM) provides a private environment for it

Virtual Machine : Guest OS Layer :

- In a virtual machine the Guest OS layer is the operating system that runs inside the virtual machine just like the OS on a physical computer it is called a guest
- Its running on top of a hypervisor which manages the hardware resources
- The Guest OS layer is what makes the VM feel like a completely separate computer while the hypervisor mimics the physical hardware like fake CPU , RAM
- The Guest OS is the software that loves top of that fake hardware
- The Guest OS acts like a normal operating system like Windows or Linux installed on a physical laptop but it thinks the virtual hardware provided by the hypervisor is real

Inside the Guest OS layer : Virtual Machine :

- The guest OS is a complete operating system and it contains all the core components you find in any OS installed on a real machine

Kernel :

- The heart of the OS and it manages the CPU , Memory and devices. In a Virtual machine , the kernel still thinks its talking to real hardware but the hypervisor gets the request and translates them to the physical hardware
- The kernel is the core of any operating system and its is the first thing to load and the last thing to shut down
- The kernel manages all the fake resources provided by the hypervisor and it decides which application gets to use the CPU and how much memory is allocated to a specific task
- In virtual machine the guest OS kernel has no idea its running on a hypervisor and thinks it is talking directly
- Being the brain of the Guest OS , it manages the virtual machine virtual CPU , Virtual memory and internal processes

Device Drivers :

- These are special programs that allow the OS to communicate with virtual hardware like virtual network , disk , and virtual CPU
- The hypervisor presents virtual devices that look like real one and so the guest OS load drivers for them like it would on a physical PC
- Even though the hardware is virtual, the OS still needs drivers to talk to it
- These are specialized pieces of software that translates something the virtual hardware understands
- Special software that allows the Guest OS to talk to fake virtual hardware such as virtual network or hard disks

System Libraries :

- Shared code like DLL on Windows that applications rely on to perform basic tasks like reading files , opening network connections and etc
- These are files like .dll in windows that contain standard instructions for common tasks like drawing a window on the screen and sending file over a network or encrypting data
- These are files that provide the basic functions and tools needed for applications to run

File System :

- The guest OS has its own file system and stored in a virtual disk file and it contains the OS files , applications and user data
- The guest OS manages its own disk space and it organizes how data is stored on the virtual hard drive
- It creates the folder structure as C:\ or /home/user and keeps track of where every bit of data is located
- The Guest OS has its own virtual hard drive where it stores its own system files , user data and configurations

System Services :

- These are programs that start automatically and run in the background without you seeing them
- With networking it keeps the VM connected to the internet and with logging it keeps the record of everything that happens in the system and the security handles logins and user permissions

Applications :

- The actual programs you want to run like web server , database or custom app and they sit on top of the OS and thus uses its kernel

Each VM need its own Guest OS :